Reliable Candidate Selection for Budget-Constrained Machine Learning Agents

DATS-SHU-200 Data Mining
Homework 3: Research Proposal

1 Introduction

Large language model (LLM) agents can automate parts of machine learning engineering, including writing training code, debugging failures, and improving existing solutions. A central difficulty is deciding which proposed solution deserves an expensive execution. For example, an agent may generate several modifications to a training pipeline, while only a small number can be trained and evaluated within the available time. Executing every proposal consumes resources; discarding proposals prematurely can remove the best solution. This project studies how an agent should spend a limited budget on comparing, testing and improving candidate programs.

Existing agent systems provide two complementary approaches to this problem. AIDE formulates machine learning engineering as tree search over code solutions: the agent drafts, debugs, and improves programs using execution feedback (Jiang et al., 2025). This makes observed performance available for subsequent decisions, but requires running candidate programs. ForeAgent instead introduces a *Predict-then-Verify* loop (Zheng et al., 2026). During improvement, it generates multiple candidates and asks an LLM to compare their code using the task description and a data analysis report. A confidence-gated selection procedure then filters candidates before execution. This approach reduces reliance on running every proposal while retaining execution feedback for the candidates that are actually verified.

Judgments can conflict, and a confident mistake can eliminate a strong program. This raises a broader question for an engineering agent. What evidence is worth acquiring before it commits training time to one solution? A judge can inspect code, the agent can consult earlier runs, or it can perform a short experiment. We will measure what each source costs and how often it improves the next choice. The first experiment uses the validation scores of programs already trained in the same run.

The baseline for this project is ForeAgent's candidate-selection component. Its released agent trajectories support a direct check of the original control logic using historical candidate batches and observations. Section 3 reports three complementary experiments: conditional historical replay, offline selection-quality evaluation, and a single-task end-to-end run. The public Preference Corpus, containing 18,438 candidate pairs across 26 tasks, supplies saved judgments and recorded execution scores for the offline evaluation (Zheng et al., 2026). MLE-bench provides the broader machine learning engineering task setting (Chan et al., 2024). Historical-output agreement, prediction accuracy, and end-to-end task performance each supply a different level of evidence.

2 Baseline Explanation

2.1 Problem formulation and notation

Let I denote a task description, D its dataset, and C a candidate program implementing a machine learning workflow. Let $M(C; I, D)$ denote its task-specific evaluation metric after execution. Define an oriented utility $u(C)$ so that higher values are always better: $u(C) = M(C; I, D)$ for a maximized metric such as AUC, and $u(C) = -M(C; I, D)$ for a minimized metric such as log loss. The

underlying engineering objective is to find

$$C^* \in \arg\max_{C \in \mathcal{C}} u(C), \tag{1}$$

where $\mathcal{C}$ is the space of possible programs. Because obtaining $u(C)$ requires execution, ForeAgent predicts relative quality before spending this cost. This notation describes the optimization objective; the true utility is unavailable to the predictor at decision time.

For candidates C_i and C_j, the comparison input consists of I, a data analysis report D_{rep}, the two programs, and a prompt P. A pretrained LLM with fixed parameters θ returns

$$(z_{ij}, \widehat{y}_{ij}, c_{ij}) = f_\theta(I, D_{\text{rep}}, C_i, C_j; P), \tag{2}$$

where z_{ij} is the generated reasoning, $\widehat{y}_{ij} \in \{0, 1\}$ identifies the preferred input position, and $c_{ij} \in [0, 1]$ is the reported confidence. The evaluation label identifies which candidate achieved the better recorded score. The index is relative to the presented pair, so it must be mapped back to candidate identity when comparisons are reused. Crucially, c_{ij} concerns this particular judgment; confidence values from unrelated pairs cannot be treated as a global ranking of candidates.

2.2 System architecture: prediction followed by verification

ForeAgent builds on AIDE's search over program nodes. Executed nodes provide code, runtime observations, and measured performance that can inform later drafting, debugging, or improvement. ForeAgent modifies the *improvement* stage by inserting prediction-based filtering between generation and execution (Zheng et al., 2026, Section 6.2):

$$\begin{aligned}\text{Executed parent} &\longrightarrow \text{Candidate generation} \longrightarrow \text{LLM comparisons} \\ &\longrightarrow \text{Confidence-gated selection} \longrightarrow \text{Execution and feedback.}\end{aligned}$$

The published setup proposes $m = 10$ improvements to a parent solution, applies a confidence threshold $\tau = 0.7$, and selects the top rank group with $k = 1$. The selected candidates are subject to the execution policy described below. Actual execution trains and evaluates a candidate program; it is distinct from calling the LLM to compare code. Successful verification returns measured feedback to the search process, so the full agent retains interaction with its computational environment.

The predictor is grounded in task and dataset context rather than code alone. The paper constructs its data reports through a profiling, verification, and verbalization pipeline: generated analysis code is executed, the resulting logs are checked, and the observations are turned into a textual report (Zheng et al., 2026, Section 3.4). The report provides information with which the LLM can assess whether a pipeline suits the data. It is part of the prediction input, not the candidate's hidden evaluation score. For the offline baseline, the released reports are reused rather than regenerated.

2.3 Inference-time selection and execution rules

The core method uses pretrained LLM inference without fitting an additional reward model or updating model parameters. Accordingly, the relevant technical specification is its decision rule, not a new training loss. The confidence gate accepts a comparison when

$$a_{ij} = \mathbf{1}\{c_{ij} \geq \tau\}, \qquad \tau = 0.7. \tag{3}$$

A high-confidence winner can advance; an inconclusive judgment does not immediately establish a loser. This is a gate on self-reported confidence, rather than a calibrated probability model fitted by this project.

Tournament aggregation. In the fixed released implementation, each round compares adjacent active candidates; an odd candidate receives a bye. Confident winners advance and losers are eliminated. Inconclusive pairs enter a pending list. Pending candidates are compared with randomly chosen advanced candidates; if none has advanced, one pending candidate advances directly. A pending winner advances without removing its already-advanced opponent; another inconclusive outcome also advances the pending candidate. The process stops at one survivor or after the round count exceeds $2m$. Final ordering prioritizes survivors, then wins and later rounds; equal consecutive $(\text{wins}, \text{round})$ groups share a displayed rank (ForeAgent authors, 2026a).

Selection and probabilistic execution. Let r_i be the zero-based displayed rank. The multi-candidate branch marks $r_i < k$ candidates eligible and independently applies an execution probability $p_{\text{exec}} = 0.2$ to each. If these tests produce an empty execution set, the original code forces execution of the first rank-0 candidate in sorted order (ForeAgent authors, 2026a). Formally, let $B_i \sim \text{Bernoulli}(p_{\text{exec}})$ and let i_0 index that first rank-0 candidate. The preliminary and final execution indicators are

$$q_i = \mathbf{1}\{r_i < k\}\, B_i, \qquad e_i = q_i + \mathbf{1}\left\{\sum_j q_j = 0\right\} \mathbf{1}\{i = i_0\}. \tag{4}$$

Thus the original multi-candidate policy executes at least one candidate, while ties may permit several executions. The probability gate and mandatory fallback together specify the execution decision used in Section 3.

2.4 Baseline evaluation and the scope of this project

The paper evaluates two different capabilities. First, its *Data-centric Solution Preference* task measures how well an LLM predicts pairwise winners. The Preference Corpus contains 895 curated solutions and 18,438 pairs from 26 tasks spanning computer vision, natural language processing, and data science. The principal metric is micro-averaged pairwise accuracy,

$$\text{Acc}_{\text{pair}} = \frac{1}{N} \sum_{\ell=1}^{N} \mathbf{1}\{\widehat{y}_\ell = y_\ell\}, \tag{5}$$

where N counts the evaluated pairs and y_ℓ is the recorded better-candidate label. The paper reports 61.5% accuracy for DeepSeek-V3.2-Thinking, averaged over three full-corpus runs (Zheng et al., 2026, Table 2). This is a published reference result, not a result achieved by this project.

Second, the full ForeAgent system is evaluated against AIDE on five MLE-bench tasks under a 12-hour limit, with three runs per task. That evaluation measures final task performance through Beat Ratio, the proportion of human leaderboard contestants outperformed, alongside execution-time and search-breadth analyses (Zheng et al., 2026, Sections 6.3–6.4). Pairwise accuracy does not establish these end-to-end outcomes: a predictor can make locally correct comparisons yet select poorly within a candidate pool, and a good offline selection does not establish actual training-time savings.

This project treats the pairwise predictor and candidate-selection controller as related baseline components. Earlier exploratory prediction runs used a fixed sample of 246 pairs across 26 tasks. The reproduction reported next evaluates the original ranking and complete execution-control logic at three levels. Historical replay checks agreement with recorded decisions, conditional on archived responses and probability outcomes. Fixed-pool offline evaluation measures the quality of the selected candidates using their recorded scores. A 12-hour Stanford COVID Vaccine run tests the complete generation, selection, execution, and feedback loop. These experiments establish implementation consistency, offline selection quality, and single-run task performance, respectively.

3 Baseline Reproduction

3.1 Objectives and experiment scope

Three experiments address complementary questions: whether the released controller reproduces archived decisions, how well it selects candidates from fixed pools, and what final task performance it achieves in a live run. All use the original implementation at commit `c4d52cf` and retain its cache behavior, confidence gate, execution probability, and mandatory execution fallback. The historical experiment is described first; the offline and online results follow in Sections 3.6 and 3.7.

The historical reproduction examines whether ForeAgent's released ranking and execution control logic reproduces its recorded historical outputs when supplied with the original candidate batches, generation order, and saved observations. The baseline is the original implementation at commit `c4d52cf`, with the original cache implementation preserved (ForeAgent authors, 2026a). The comparison backend supplies recorded responses by batch position, and the execution branch receives the probability outcomes recorded in the logs. The resulting experiment is a *conditional historical-trace replay*: it verifies the state updates produced from those observations.

The experiment is frozen as `foreagent_original_historical_replay_v1`, dated November 7, 2026. The freeze record binds the source snapshots, input manifests, scripts, verification outputs, and archive using SHA-256 checksums. Subsequent cache corrections and alternative aggregation methods belong to separately versioned experiments. The frozen historical result concerns agreement with the original control logic; the later subsections assess selection quality and live task performance separately.

3.2 Released records and experimental setup

The inputs come from the authors' public `agent_runs/ForeAgent/` release, fixed at dataset revision `6b322cb` (ForeAgent authors, 2026b). All 15 released ForeAgent runs were inspected: three runs for each of five MLE-bench tasks. For each run, the inputs include its actual `config.yaml`, full `journal.json`, and full `aide.log`. These 45 original text files were checked against their recorded sizes and hashes. Table 1 summarizes recovery coverage.

Table 1: Coverage of the released historical runs. Logged batches include interrupted generation and batches with final-journal gaps. Each recovered batch contains ten candidates.

Task	Runs	Logged batches	Recovered	Candidates
Aerial Cactus Identification	3	14	12	120
Histopathologic Cancer Detection	3	5	4	40
Stanford COVID Vaccine	3	45	42	420
Statoil Iceberg Classification	3	60	57	570
Ventilator Pressure Prediction	3	16	14	140
Total	15	140	129	1,290

The saved configurations agree on the relevant settings: ten improvement candidates, confidence threshold 0.7, Top-$k = 1$, execution probability 0.2, and a 12-hour agent time limit. The comparison-model label is DeepSeek-V3.2-Thinking with temperature 1.0. These model settings describe the source runs; this replay consumes their saved responses. Local callbacks record execution decisions, while training and inference observations come from the archived runs. Additional model API calls and candidate-training runs both equal zero.

Recovery uses record availability as the eligibility criterion. Of 140 logged batches, 129 have completed comparison histories and corresponding final-journal nodes. The remaining 11 are retained in a separate coverage ledger: six end during generation, and five contain generated nodes absent from the final journal. This rule was applied using structural completeness, independently of candidate scores or agreement results.

3.3 Recovery and replay procedure

Candidate identity, grouping, and order. Messages of the form `Generated improvement candidate i/10` identify the original generation order. Candidate IDs are resolved to their full journal entries and actual source code. The `node2parent` map verifies a shared parent within each recovered batch. Creation-time ordering provides a second check on the logged sequence. The journal's stored list reflects post-selection saving and re-appending, so generation logs and timestamps establish the replay input order.

Historical comparisons. Each candidate in a batch stores the same `wm_comparison_results` sequence, containing winner position, confidence, round, and validity. A local provider supplies these archived results to the original `tournament_rank()` calls in their recorded order. The replay checks response counts, round indices, and agreement between confidence and the 0.7 validity gate. It also checks 8,725 ordinary comparisons against the logged participant IDs and outcomes, and 15,148 revival comparisons against the logged pending-candidate identities and outcomes.

The original controller's random seed or RNG state remains an open recovery item. For revival matches, the advanced opponent is represented by an explicitly marked placeholder choice generated with demonstration seed 0. Responses are supplied by recorded batch position. The checked historical information identifies the pending candidate and outcome; it supplies partial evidence about the

advanced opponent. Consequently, the agreement claim concerns control-flow replay conditional on the recorded responses, with exact original revival-pair reconstruction reserved for further log analysis.

Execution decisions and mandatory fallback. The original execution statements are extracted directly from `agent.py`, lines 700–767, and checked for AST equality. Logged probability-trigger outcomes drive the eligible candidates' execution tests. The original code then applies its mandatory fallback: whenever this stage selects an empty execution set, the first rank-0 candidate in sorted order is executed. Local callbacks record selected identities. The resulting execution set is checked against journal `is_skipped` flags, and its order is checked against execution-parsing log messages. Supplying observed probability outcomes conditions this check on the historical draws; the verified behavior is Top-k handling, candidate selection, and fallback processing.

3.4 Results and agreement with the released traces

For all 129 recovered batches, the original implementation reproduces the recorded candidate ranks, round metadata, sorted order, and final execution decisions. Table 2 reports the comparison units explicitly. Across these batches, the replay consumes 23,873 comparison records, including repeated rounds and cached observations. These records are observations from the historical trajectory, with repeated comparisons contributing to the recorded control flow.

Table 2: Agreement conditional on archived comparison responses and logged probability outcomes. Candidate-level and batch-level denominators measure different units.

Checked output	Agreement	Rate
Candidate displayed rank	1,290 / 1,290	100%
Candidate round metadata	1,290 / 1,290	100%
Complete sorted candidate order	129 / 129 batches	100%
Final execution set	129 / 129 batches	100%
Final execution order	129 / 129 batches	100%

Mandatory execution fallback occurs in 87 of the recovered batches, matching the log events. For example, in the first recovered batch of the Aerial Cactus run with UUID prefix `c5d8f2ae`, candidate `69daaf39` receives rank 0. Its probability test is recorded as skipped, after which the original fallback selects that candidate. Both the resulting execution identity and the final journal flag agree with the replay.

The 100% values represent *historical-output agreement*. The recorded winner positions are supplied as observations; true candidate-performance scores play no role in recovering ranks. Thus the result establishes faithful processing of these observed histories. Pairwise prediction accuracy, selection quality against executed scores, and end-to-end agent efficiency are separate evaluation questions.

3.5 Interpretation and reproducibility artifacts

The replay establishes an implementation baseline for the candidate-selection component: the recovered historical observations lead to the same ranking and execution outputs under the released control rules. The paper's full-corpus prediction accuracy and its five-task online performance results provide broader reference experiments, while this section contributes a direct consistency check against the released agent trajectories (Zheng et al., 2026). The archived responses already reflect the source runs' inference and cache behavior; fresh model judgments and original revival-opponent reconstruction require their own experimental conditions.

The accompanying `baseline_freeze.json` pins the source, data, scripts, and verified outputs by SHA-256. The archive contains full logs and journals, the coverage ledger, and per-batch comparisons. Its `replay_author.py` and `verify_and_report.py` scripts reproduce and check these results.

3.6 Offline selection quality: `corpus26-v2`

Setup. This experiment asks whether the original tournament selects candidates with better recorded execution scores than uniform random choice. It uses candidates and one set of published pairwise

judgments from the Preference Corpus. Each task contributes 100 candidate-pool instances, each replayed three times with fixed orders and seeds: 2,600 pool instances, 2,164 distinct candidate sets, and 7,800 original-controller replays across 26 tasks. Twenty-one tasks use pools of ten candidates; the other five use pools of 3, 3, 5, 6, and 8. Pool construction uses finite-score availability and pairwise-feedback coverage; score magnitude and candidate rank are reserved for evaluation after selection.

The settings are confidence threshold 0.7, Top-$k = 1$, and execution probability 0.2. The original cache is retained and cleared before each replay; comparisons run serially for deterministic replay. Saved judgments replace new LLM responses, and candidate filenames serve as identity stubs in the cache. Actual execution is represented by recording selected identities and looking up their existing scores. Both the probability stage and the original mandatory fallback are included. Additional API calls and candidate-training runs equal zero.

Metrics. Let S be the displayed rank-0 group, $B = \arg\max_{i \in P} u_i$ the best candidates in pool P, and E the final simulated execution set. The selection metric averages a uniform single choice within S:

$$H = \frac{|S \cap B|}{|S|}, \qquad R = \frac{\max_{i \in P} u_i - |S|^{-1} \sum_{i \in S} u_i}{\max_{i \in P} u_i - \min_{i \in P} u_i}. \tag{6}$$

Here R is normalized regret for a pool with a nonzero utility range; lower is better. The random reference chooses uniformly from the same pool, with hit probability $|B|/|P|$. We average repeats within a pool, pools within a task, and then the 26 tasks equally. The execution diagnostic is $\mathbf{1}\{E \cap B \neq \varnothing\}$ under the complete original policy. This separates single-choice selection quality from a policy that may execute several tied candidates.

Table 3: Original ForeAgent offline selection quality on the fixed corpus pools. Values are task-macro averages. Uniform random choice uses the same candidate pools.

Metric	Original ForeAgent	Uniform random choice
Uniform-tie best-hit (H, higher better)	20.43%	12.53%
Normalized regret (R, lower better)	0.2810	0.3674

Results and interpretation. Table 3 shows a descriptive 7.89 percentage-point advantage in best-hit probability over random choice, together with lower regret. The complete execution simulation includes a best candidate in 21.60% of replays and executes 1.028 candidates per replay on average. Every replay has a final execution; the mandatory fallback is used in 74.05% of replays. These values describe this fixed-feedback offline experiment.

The task pools overlap and reuse judgments, so the 7,800 replays are dependent observations. Candidates are assembled from the corpus rather than recovered as siblings generated in one live improvement step. Benchmark judgments and the serial, identity-stub adaptation also differ from fresh judgments under the live Agent prompt. The results therefore support a fixed-pool selection comparison; live search efficiency and robustness across fresh judgments have their own evaluation scope. The original result is retained separately from cache-corrected and alternative-ranking experiments. Evidence is in `experiments/corpus26-v2/`, including the protocol, complete-execution specification, original results, and verification records. Earlier probability-stage-only execution results are archived as superseded; the table uses the complete-policy version.

3.7 Single-task end-to-end reproduction

Task and environment. Run `stanford-autodl-v2` executes the original ForeAgent on Stanford COVID Vaccine. The task predicts RNA degradation-related measurements. The prepared data contain 2,160 training sequences and 240 test sequences; the Kaggle archive and four prepared files match the fixed release's recorded MD5 checksums. The released task data report is reused. The runtime uses the pinned author OCI image through PRoot on an AutoDL RTX 4090, with the controller and model requests on a Mac. DeepSeek-V3.2 generates code, DeepSeek-V3.2-Thinking compares candidates, and `gpt-4o-mini-2024-07-18` parses execution feedback. The DeepSeek requests use the SiliconFlow service with model-name and thinking-mode mapping; the feedback requests use the official OpenAI endpoint.

The run retains ten improvement candidates, confidence 0.70, Top-$k = 1$, execution probability 0.20, and the original cache and fallback. The 43,200-second wall-clock budget ran from November 8, 2026, 01:22:02 to 13:22:02, China Standard Time. Generation, API waits and retries, selection, execution, and feedback all consume this budget. The original algorithm selects its final submission by its internal validation metric. External evaluation uses the official private-answer scorer after the run.

Run outputs. The completed journal contains 82 nodes: 22 executed and 60 skipped. There are 19 completed outer iterations and seven complete tournament batches. An outer iteration may generate several nodes; these counts use different units. The retained node is `84c79a8d`, with parsed internal validation value 0.4378. Its submission contains 25,680 valid prediction IDs. The official private-set error is **0.26640**; an independent, ID-aligned calculation gives 0.2663989312. The scored subset contains 16,320 sequence-position rows. The metric is mean columnwise root mean squared error (MCRMSE) across reactivity, deg_Mg_pH10, and deg_Mg_50C; lower is better. The internal validation value and private-set score represent separate evaluations.

Table 4: Same-task endpoint comparison. The authors' three released final submissions and this run are scored against the same private answers. Each entry is a single run; lower MCRMSE is better.

Stanford COVID Vaccine submission	Private-set MCRMSE
Author run `01abb886`	0.25768
Author run `5d12c313`	0.24320
Author run `cbb2a0b0`	0.27497
This reproduction (one 12-hour run)	**0.26640**

Comparison and operating conditions. The score lies within the range of the authors' three same-task endpoints (Table 4); their mean is 0.25862. The paper evaluates five tasks with three runs per task, whereas this reproduction contributes one run on one of those tasks. This provides evidence that the complete original loop produces a valid submission with a comparable same-task score. Repeated-run variability and the online contribution of filtering require repeated runs and a matched execution-based baseline.

Hardware, model services, and network latency differ from the historical runs. After the run, the experimenter reported that Mac screen sleep and VPN disconnection disrupted model API access. One feedback request, including retries, took 13,491.88 seconds (about 3 hours 45 minutes); this elapsed request time includes processing and retries as well as interruption-related waiting. The entire interval counts toward the standard 12-hour budget. The saved evidence supplies the request elapsed time; the VPN outage duration has a separate measurement gap. This operating condition reduces the time available for productive search and is retained in the reported result. A separately initiated extension was stopped at the user's request with zero new completed nodes; its retained prediction hash matches the standard submission. Table 4 reports the standard 12-hour outcome.

Scoring and artifact checks. The original scorer and an independent ID-aligned MCRMSE calculation agree for this submission and all three author submissions. A row-shuffle diagnostic revealed index-sensitive behavior in the original pandas-based scorer: shuffling the same predictions changes its score, while ID-aligned evaluation remains invariant. The archived inputs have been checked for unique IDs, required columns, finite predictions, and agreement between both scoring methods. The original scorer is preserved, and the verification records make its ordering condition explicit.

The source check verifies 484 Git files against the pinned commit. The final prediction file is tied to the retained journal node and archived with its SHA-256 checksum. Run counts, timing, submission hashes, and author references are recorded in `experiments/online-stanford-v2/`; the independent source, data, artifact, and scoring checks are recorded in `experiments/environment-acceptance/`. Together with historical replay and fixed-pool evaluation, these results establish three distinct pieces of baseline evidence: faithful control processing, measurable offline selection quality, and a completed single-task Agent run.

4 Future Plan

Research objective. Within a fixed time and compute budget, which observations help an agent choose what to run or improve next? We will evaluate immediate choices against measured candidate performance and evaluate full runs by their final submission scores and costs. C0, our corrected version of the original tournament, provides the control for method changes. The Stanford reproduction in Section 3.7 used the original agent.

First experiment. Use past executions to choose the next program. For each candidate, retrieve the three most similar programs among up to ten earlier executions from the same run, using TF–IDF cosine similarity over code tokens and adjacent token pairs. Weight the neighbours' validation ranks by their similarity. Combine the resulting candidate rank with its judge rank, giving each equal weight. The judge rule prioritizes the C0 top group, then accepted win fraction and number of accepted opponents. With fewer than three usable earlier scores, both methods use C0.

In a pilot on 26 development tasks, programs from outside the current pool and their corpus scores stood in for a live agent's execution history. Equal-weight fusion, a prespecified secondary method, lowered task-mean normalized regret from 0.2642 to 0.1966 with DeepSeek judgments and from 0.2725 to 0.2030 with pooled GPT judgments. Breaking only C0 ties, the primary method, gained 0.0093 on GPT against a 0.01 advancement criterion. The next test uses validation scores actually available at selection time.

CPU screen on real search histories. We checked 136 batches of ten siblings from 16 runs across five tasks. At least three earlier validation scores were available in 130 batches and ten were available in 60. Only six batches offered enough history and two or more executed siblings with distinct comparable outcomes. On their 13 pairs, memory alone ordered 29.6% correctly under task-balanced averaging, versus 50.8% after shuffling history scores. The original tournament tied on every observed pair.

We replayed C0 with the archived parsed judgments and a cap of 13 distinct feedback reads. Its top group could be reconstructed in **128 of 136 batches**. Using the same fixed tie order for both selectors in each batch, fusion changed the chosen program in **80 of 128 (62.5%)**. In 47 of these batches the top groups were disjoint, so every tie order changes the choice. Six other batches contained duplicate programs outside the C0 replay interface. The two remaining batches required unobserved judgments. In one Aerial Cactus batch all admissible completions implied a switch, even though the baseline top group was uncertain. Thus among the 130 eligible batches we established **81 switches, 48 agreements and one unresolved choice**. Among the 80 observed choice changes, zero pairs had usable historical validation scores for both selected programs. Paired execution supplies the missing outcomes.

Measure the value of a changed choice. Freeze the selectors, task sample, scorer, seeds and training limits before collecting outcomes. Give C0 and fusion the same candidates and comparisons and record the program each would run. Each agreement contributes zero score difference. At disagreements train both choices on the same split and budget. Report the disagreement rate over all batches, the paired score change within each task, failures and elapsed time. Repeat a subset with another seed to estimate training variability. A control that shuffles historical scores tests how much the association between a program and its score contributes. Charge runs made solely to evaluate a counterfactual choice to the evaluation budget. If the paired gains recur across tasks and exceed training variation at comparable cost, freeze the method for full agent runs on fresh tasks.

Other directions. Three further directions address different decisions. If retrieval scores prove weak, supply the pairwise judge with scored examples and measure how well it predicts execution outcomes relative to the original prompt. At uncertain decisions, compare spending the remaining budget on another code comparison with running a short training test. If the best candidate in a batch already performs poorly, vary which parent is expanded or how changes to its code are proposed, keeping the number of generated programs and training budget fixed. Each test measures the quality of the selected program and the cost of obtaining it.

References

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025. https://arxiv.org/abs/2502.13138.

Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. Can We Predict Before Executing Machine Learning Agents? In *Proceedings of ACL 2026 (Volume 1: Long Papers)*, pp. 3941–3974, 2026. https://aclanthology.org/2026.acl-long.182/.

Jun Shern Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *arXiv:2410.07095*, 2024. https://arxiv.org/abs/2410.07095.

ForeAgent authors. Released implementation of Predict Before Execute. GitHub repository, fixed commit c4d52cf (full identifier in the URL below). Relevant files: world_model.py, agent.py, and config.yaml under mle-bench/agents/aide/. https://github.com/zjunlp/predict-before-execute/tree/c4d52cf99bd870d830b456ac7c0684aec1aef375.

ForeAgent authors. PredictBeforeExecute: released ForeAgent execution records. Hugging Face dataset, fixed revision 6b322cb; subdirectory agent_runs/ForeAgent/. https://huggingface.co/datasets/zjunlp/PredictBeforeExecute/tree/6b322cb88bdbcb2b2d3897ec7d0ded94a5bb2d06/agent_runs/ForeAgent.

Generative AI use. OpenAI Codex assisted with source and log inspection, Python replay and verification tooling, English drafting, and LaTeX typesetting. Requests included (translated from Chinese): "Replay the original version using the authors' real candidate groups, order, and saved judgments, and check agreement" and "Freeze this part and add it to HW3's Baseline Reproduction using the previous NeurIPS template." A further request was to incorporate the corpus26-v2 offline selection results and the new end-to-end run while retaining the 129-batch historical replay. Original prompts and affected components are recorded in the accompanying AI_USAGE.txt.

Claude Code assisted with the corpus pilot, and OpenAI Codex discussed the plan with Claude in an existing Claude Desktop chat. Codex checked the pilot summaries, wrote the chronological replay and selection audit, and drafted Section 4. The follow-up request was to keep the plan broad, list execution memory first, and use subagents to check on CPU how many of the 136 batches support a C0 reconstruction and how often adding memory changes the choice (translated from Chinese).